

These three concepts are among the most important topics in JavaScript because they explain **how variables are stored and accessed during program execution**.

1. Scope

Scope determines where a variable can be accessed in your code.

Think of scope as a room.

- Variables inside a room can only be used in that room.
- Variables outside the room cannot enter unless they are visible from there.

Types of Scope

Global Scope

Variables declared outside all functions and blocks.

JavaScript



```
let college = "HIT";

function showCollege() {
  console.log(college);
}

showCollege();
```

Output:

JavaScript



HIT

`college` can be accessed from anywhere.

Function Scope

Variables declared inside a function are only accessible within that function.

JavaScript



```
function greet() {
  let name = "Shivam";
  console.log(name);
}
```

```
}

greet();

console.log(name); // Error
```

Output:

JavaScript



```
Shivam
ReferenceError
```

Block Scope

Variables declared with `let` and `const` are block scoped.

JavaScript



```
if (true) {
  let age = 21;
  console.log(age);
}

console.log(age); // Error
```

Output:

JavaScript



```
21
ReferenceError
```

var vs let

JavaScript



```
if (true) {
  var x = 10;
  let y = 20;
}

console.log(x); // 10
console.log(y); // Error
```

`var` ignores block scope while `let` respects it.

2. Execution Context

Whenever JavaScript runs code, it creates an **Execution Context**.

You can think of it as a **workspace** where JS keeps:

- Variables
 - Functions
 - The value of `this`
-

Types of Execution Context

Global Execution Context (GEC)

Created when the program starts.

JavaScript



```
let name = "Shivam";

function hello() {
  console.log("Hello");
}
```

JS first creates a Global Execution Context.

Function Execution Context (FEC)

Created whenever a function is called.

JavaScript



```
function greet() {
  let msg = "Hi";
  console.log(msg);
}

greet();
```

When `greet()` runs, JS creates a new execution context for it.

Execution Context Phases

Phase 1: Memory Creation Phase

JavaScript scans the code before executing it.

Memory is allocated:

```
JavaScript
console.log(a);

var a = 10;
```

Memory Phase:

```
JavaScript
a = undefined
```

Phase 2: Execution Phase

Code executes line by line.

```
JavaScript
a = 10
```

Now `a` gets its actual value.

Example

```
JavaScript

var x = 10;

function test() {
  var y = 20;
  console.log(y);
}

test();
```

Memory Phase

JavaScript



```
x = undefined
test = function
```

Execution Phase

JavaScript



```
x = 10
test() called
```

Function context created:

JavaScript



```
y = undefined
```

Then:

JavaScript



```
y = 20
console.log(y)
```

Output:

JavaScript



```
20
```

Call Stack and Execution Context

JavaScript manages execution contexts using a stack.

JavaScript



```
function one() {
  two();
}

function two() {
  three();
}

function three() {
```

```
    console.log("Done");  
  }  
  
  one();
```

Stack Flow

```
Global  
↓  
one()  
↓  
two()  
↓  
three()
```



Then they are removed in reverse order:

```
three()  
two()  
one()  
Global
```



This is called the **Call Stack**.

3. Closures

A Closure is created when a function remembers variables from its outer scope even after the outer function has finished execution.

This is one of JavaScript's most powerful features.

Basic Example

</> JavaScript



```
function outer() {  
  let count = 0;  
  
  function inner() {  
    count++;  
    console.log(count);  
  }  
  
  return inner;
```

```
}

const counter = outer();

counter();
counter();
counter();
```

Output:

JavaScript

```
1
2
3
```

What Happened?

Normally, after `outer()` finishes, its variables should disappear.

But `inner()` still needs `count`.

So JavaScript keeps `count` alive.

This preserved relationship is called a **Closure**.

Visual Representation

```
outer()
|
├─ count = 0
|
└─ returns inner()
    |
    └─ remembers count
```

Even after `outer()` ends:

```
count still exists
```

because `inner()` references it.

Real-Life Analogy

Imagine a teacher writes marks in a notebook.

After class ends, students leave.

But the notebook remains.

Next day the teacher can still see the marks.

```
Teacher = inner function
Notebook = closure memory
Marks = variables
```



The data survives because someone still has access to it.

Practical Example: Private Variables

<> JavaScript



```
function bankAccount() {
  let balance = 1000;

  return {
    deposit(amount) {
      balance += amount;
    },

    getBalance() {
      return balance;
    }
  };
}

const account = bankAccount();

account.deposit(500);

console.log(account.getBalance());
```

Output:

<> JavaScript



```
1500
```

Notice:

JavaScript



```
console.log(account.balance);
```

Output:

JavaScript



```
undefined
```

The balance is private and accessible only through closures.

Interview Question

What will be the output?

JavaScript



```
function outer() {  
  let x = 10;  
  
  function inner() {  
    console.log(x);  
  }  
  
  return inner;  
}  
  
const fn = outer();  
fn();
```

Output:

JavaScript



```
10
```

Because `inner()` remembers `x`.

Relationship Between All Three

JavaScript



```
function outer() {  
  let x = 10;  
  
  function inner() {  
    console.log(x);  
  }  
  
  return inner;  
}  
  
const fn = outer();  
fn();
```

Scope

`inner()` can access variables from `outer()` because of lexical scope.

Execution Context

When `outer()` runs, JS creates an execution context containing `x`.

Closure

When `outer()` finishes, `inner()` still remembers `x`.

One-Line Summary

- **Scope** → Where variables are accessible.
- **Execution Context** → The environment created when code runs.
- **Closure** → A function remembering variables from its outer scope even after that scope has finished executing.

If you're preparing for interviews, the next concepts to learn after these are **Hoisting**, **Lexical Environment**, **Event Loop**, **Call Stack**, **Promises**, and `this` keyword, because they build directly on Scope and Execution Context.